

Python List Comprehensions

August 8, 2026

Contents

1	Filter Words That Contain a Digit	2
2	Reverse Words Longer Than Four Characters	3
3	Square Even, Cube Odd (Positive Numbers Only)	4
4	Case-Insensitive Palindrome Check	6
5	Uppercase Vowels, Lowercase Consonants	7
6	Words With Matching First and Last Letters	9
7	Remove Duplicates While Preserving Order	10
8	Keep Purely Alphabetic Words, Lowercased	11
9	Sort Words by Length (Descending)	12
10	First-and-Last-Letter Abbreviation	13

Problem 1. Filter Words That Contain a Digit

1.1 Code

```
1 words = ["abc", "a1b", "xyz9", "hello "]
2 result = [w for w in words if any(ch.isdigit() for ch in w)]
3 print(result)
```

Step-by-Step Trace

Step 1: Start the list comprehension. We loop through each element of `words`. The variable `w` will take each word one by one.

Iteration 1:

`w = "abc"`

- Check `any(ch.isdigit() for ch in w)`:
 - `ch = "a" → "a".isdigit() → False`
 - `ch = "b" → "b".isdigit() → False`
 - `ch = "c" → "c".isdigit() → False`
- Result: `any(False, False, False) → False`
- Condition fails → `"abc"` is **not included** in result.

Iteration 2:

`w = "a1b"`

- Check `any(ch.isdigit() for ch in w)`:
 - `ch = "a" → False`
 - `ch = "1" → True`
 - `ch = "b" → False`
- Result: `any(False, True, False) → True`
- Condition passes → `"a1b"` is **included** in result.

Iteration 3:

`w = "xyz9"`

- Check `any(ch.isdigit() for ch in w)`:
 - `ch = "x" → False`
 - `ch = "y" → False`
 - `ch = "z" → False`
 - `ch = "9" → True`
- Result: `any(False, False, False, True) → True`
- Condition passes → `"xyz9"` is **included** in result.

Iteration 4:

`w = "hello "`

- Check `any(ch.isdigit() for ch in w)`:
 - `ch = "h" → False`
 - `ch = "e" → False`
 - `ch = "l" → False`
 - `ch = "l" → False`

- `ch = "o" → False`
- `ch = " " (space) → False`
- Result: `any(False, False, False, False, False, False) → False`
- Condition fails → `"hello "` is **not included** in result.

Step 2: Collect results. After all iterations, the words that passed the condition are

```
result = ["a1b", "xyz9"]
```

Step 3: Print output. `print(result)` produces the final output shown below.

1.2 Final Output

```
1 print(result)
2 # Output: ['a1b', 'xyz9']
```

Problem 2. Reverse Words Longer Than Four Characters

2.1 Code

```
1 sentence = "Python makes programming enjoyable"
2 words = sentence.split()
3 result = [w[::-1] for w in words if len(w) > 4]
4 print(result)
```

Step-by-Step Trace

Step 1: Split the sentence. `sentence.split()` breaks the string into a list of words, separated by spaces, so

```
words = ["Python", "makes", "programming", "enjoyable"]
```

Step 2: List comprehension begins. We loop through each word in `words`. The variable `w` will take each word one by one.

Iteration 1:

```
w = "Python"
```

- Check condition: `len("Python") = 6 → greater than 4 → condition passes.`
- Reverse word: `"Python"[::-1] = "nohtyP".`
- `"nohtyP"` is added to `result`.

Iteration 2:

```
w = "makes"
```

- Check condition: `len("makes") = 5` → greater than 4 → condition passes.
- Reverse word: `"makes"[::-1] = "sekam"`.
- `"sekam"` is added to result.

Iteration 3:

`w = "programming"`

- Check condition: `len("programming") = 11` → greater than 4 → condition passes.
- Reverse word: `"programming"[::-1] = "gnimmargorp"`.
- `"gnimmargorp"` is added to result.

Iteration 4:

`w = "enjoyable"`

- Check condition: `len("enjoyable") = 9` → greater than 4 → condition passes.
- Reverse word: `"enjoyable"[::-1] = "elbayojne"`.
- `"elbayojne"` is added to result.

Step 3: Collect results. After all iterations, the list is

```
result = ["nohtyP", "sekam", "gnimmargorp", "elbayojne"]
```

Step 4: Print output. `print(result)` produces the final output shown below.

2.2 Final Output

```
1 print(result)
2 # Output: ['nohtyP', 'sekam', 'gnimmargorp', 'elbayojne']
```

Problem 3. Square Even, Cube Odd (Positive Numbers Only)

3.1 Code

```
1 nums = [-3, -2, 1, 2, 3, 4]
2 result = [
3     n*n if n % 2 == 0 else n**3
4     for n in nums
5     if n > 0
6 ]
7 print(result)
```

Step-by-Step Trace

Step 1: Input list. We start with `nums = [-3, -2, 1, 2, 3, 4]`.

Step 2: List comprehension begins. We loop through each number `n` in `nums`. The condition `if n > 0` ignores all negative numbers.

Iteration 1:

`n = -3`

- Condition check: `n > 0` $\rightarrow$ False.
- Skip this number.

Iteration 2:

`n = -2`

- Condition check: `n > 0` $\rightarrow$ False.
- Skip this number.

Iteration 3:

`n = 1`

- Condition check: `n > 0` $\rightarrow$ True.
- Apply transformation: `n % 2 == 0?` $\rightarrow 1 \% 2 = 1 \rightarrow$ odd.
- So use `n**3 = 1**3 = 1`.
- Add 1 to result.

Iteration 4:

`n = 2`

- Condition check: `n > 0` $\rightarrow$ True.
- Transformation: `2 % 2 = 0` $\rightarrow$ even.
- So use `n*n = 2*2 = 4`.
- Add 4 to result.

Iteration 5:

`n = 3`

- Condition check: `n > 0` $\rightarrow$ True.
- Transformation: `3 % 2 = 1` $\rightarrow$ odd.
- So use `n**3 = 3**3 = 27`.
- Add 27 to result.

Iteration 6:

`n = 4`

- Condition check: `n > 0` $\rightarrow$ True.
- Transformation: `4 % 2 = 0` $\rightarrow$ even.
- So use `n*n = 4*4 = 16`.
- Add 16 to result.

Step 3: Collect results. After all iterations: `result = [1, 4, 27, 16]`.

Step 4: Print output. `print(result)` produces the final output shown below.

3.2 Final Output

```
1 print(result)
2 # Output: [1, 4, 27, 16]
```

Problem 4. Case-Insensitive Palindrome Check

4.1 Code

```
1 words = ["Level", "python", "Radar", "hello"]
2 result = [w for w in words if w.lower() == w.lower()[::-1]]
3 print(result)
```

Step-by-Step Trace

Step 1: Input list. We start with `words = ["Level", "python", "Radar", "hello"]`.

Step 2: List comprehension begins. We loop through each word `w` in `words`. The condition `w.lower() == w.lower()[::-1]` checks if the word, converted to lowercase, is equal to its reversed version. If true, the word is a palindrome (case-insensitive).

Iteration 1:

`w = "Level"`

- Convert to lowercase: `"Level".lower() → "level"`.
- Reverse: `"level"[::-1] = "level"`.
- Compare: `"level" vs "level" → equal`.
- Condition passes → `"Level"` is **included**.

Iteration 2:

`w = "python"`

- Lowercase: `"python"`.
- Reverse: `"python"[::-1] = "nohtyp"`.
- Compare: `"python" vs "nohtyp" → not equal`.
- Condition fails → `"python"` is **not included**.

Iteration 3:

`w = "Radar"`

- Lowercase: `"radar"`.
- Reverse: `"radar"[::-1] = "radar"`.
- Compare: `"radar" vs "radar" → equal`.
- Condition passes → `"Radar"` is **included**.

Iteration 4:

`w = "hello"`

- Lowercase: `"hello"`.

- Reverse: `"hello"[::-1] = "olleh"`.
- Compare: `"hello"` vs `"olleh"` → not equal.
- Condition fails → `"hello"` is **not included**.

Step 3: Collect results. After all iterations: `result = ["Radar"]`.

Step 4: Print output. `print(result)` produces the final output shown below.

4.2 Final Output

```
1 print(result)
2 # Output: ['Level', 'Radar']
```

Problem 5. Uppercase Vowels, Lowercase Consonants

5.1 Code

```
1 s = "a1B2cDe3"
2 result = [
3     ch.upper() if ch.lower() in "aeiou" else ch.lower()
4     for ch in s
5     if ch.isalpha()
6 ]
7 print(result)
```

Step-by-Step Trace

Step 1: Input string. We start with `s = "a1B2cDe3"`.

Step 2: List comprehension begins. We loop through each character `ch` in the string `s`. The condition `if ch.isalpha()` ignores digits and keeps only alphabetic characters. The transformation converts a character to uppercase if it is a vowel (a, e, i, o, u), and to lowercase otherwise (consonants).

Iteration 1:

`ch = "a"`

- Check: `"a".isalpha()` → True.
- `"a".lower() = "a"` → vowel.
- Transformation: `"a".upper()` → `"A"`.
- Add `"A"` to `result`.

Iteration 2:

`ch = "1"`

- Check: `"1".isalpha()` → False.
- Skip this character.

Iteration 3:

ch = "B"

- Check: "B".isalpha() → True.
- "B".lower() = "b" → consonant.
- Transformation: "B".lower() → "b".
- Add "b" to result.

Iteration 4:

ch = "2"

- Check: "2".isalpha() → False.
- Skip this character.

Iteration 5:

ch = "c"

- Check: "c".isalpha() → True.
- "c".lower() = "c" → consonant.
- Transformation: "c".lower() → "c".
- Add "c" to result.

Iteration 6:

ch = "D"

- Check: "D".isalpha() → True.
- "D".lower() = "d" → consonant.
- Transformation: "D".lower() → "d".
- Add "d" to result.

Iteration 7:

ch = "e"

- Check: "e".isalpha() → True.
- "e".lower() = "e" → vowel.
- Transformation: "e".upper() → "E".
- Add "E" to result.

Iteration 8:

ch = "3"

- Check: "3".isalpha() → False.
- Skip this character.

Step 3: Collect results. After all iterations: `result = ["A", "b", "c", "d", "E"]`.

Step 4: Print output. `print(result)` produces the final output shown below.

5.2 Final Output

```
1 print(result)
```



```
2 # Output: ['A', 'b', 'c', 'd', 'E']
```

Problem 6. Words With Matching First and Last Letters

6.1 Code

```
1 words = ["level", "test", "radar", "python"]
2 result = [(w, len(w)) for w in words if w[0] == w[-1]]
3 print(result)
```

Step-by-Step Trace

Step 1: Input list. We start with `words = ["level", "test", "radar", "python"]`.

Step 2: List comprehension. We loop through each word `w`. The condition `w[0] == w[-1]` checks if the first and last characters are the same. If true, `(w, len(w))` is added to `result`.

Iteration 1:

`w = "level"`

- First character: "l".
- Last character: "l".
- Compare: "l" == "l" → True.
- Add tuple ("level", 5).

Iteration 2:

`w = "test"`

- First character: "t".
- Last character: "t".
- Compare: "t" == "t" → True.
- Add tuple ("test", 4).

Iteration 3:

`w = "radar"`

- First character: "r".
- Last character: "r".
- Compare: "r" == "r" → True.
- Add tuple ("radar", 5).

Iteration 4:

`w = "python"`

- First character: "p".
- Last character: "n".
- Compare: "p" == "n" → False.
- Skip.

Step 3: Collect results.

```
result = [("level", 5), ("test", 4), ("radar", 5)]
```

Step 4: Print output. `print(result)` produces the final output shown below.

6.2 Final Output

```
1 print(result)
2 # Output: [('level', 5), ('test', 4), ('radar', 5)]
```

Words starting and ending with the same character are "level", "test", and "radar", with lengths 5, 4, and 5 respectively. "python" is excluded because it starts with "p" and ends with "n".

Problem 7. Remove Duplicates While Preserving Order

7.1 Code

```
1 nums = [1, 2, 2, 3, 1, 4, 3]
2 result = []
3 [result.append(n) for n in nums if n not in result]
4 print(result)
```

Step-by-Step Trace

Step 1: Input list. `nums = [1, 2, 2, 3, 1, 4, 3]`.

Step 2: Initialize result. `result = []` starts as an empty list.

Step 3: List comprehension with append. We loop through each number `n` in `nums`. The condition `if n not in result` appends `n` only if it has not already been added, so duplicates are skipped while the first occurrence is kept.

Iteration 1:

`n = 1`

- Check: `1 not in result` → True (result is empty).
- Append: `result = [1]`.

Iteration 2:

`n = 2`

- Check: `2 not in result` → True.
- Append: `result = [1, 2]`.

Iteration 3 (again):

`n = 2`

- Check: `2 not in result` → False (already present).
- Skip.

Iteration 4:

`n = 3`

- Check: `3 not in result` → True.
- Append: `result = [1, 2, 3]`.

Iteration 5 (again):

`n = 1`

- Check: `1 not in result` → False (already present).
- Skip.

Iteration 6:

`n = 4`

- Check: `4 not in result` → True.
- Append: `result = [1, 2, 3, 4]`.

Iteration 7 (again):

`n = 3`

- Check: `3 not in result` → False (already present).
- Skip.

Step 4: Final result. `result = [1, 2, 3, 4]`.

Step 5: Print output. `print(result)` produces the final output shown below.

7.2 Final Output

```
1 print(result)
2 # Output: [1, 2, 3, 4]
```

Problem 8. Keep Purely Alphabetic Words, Lowercased

8.1 Code

```
1 words = ["Hello", "World123", "Python", "AI!"]
2 result = [w.lower() for w in words if w.isalpha()]
3 print(result)
```

Step-by-Step Trace

Step 1: Input list. `words = ["Hello", "World123", "Python", "AI!"]`.

Step 2: List comprehension. We loop through each word `w`. The condition `w.isalpha()` keeps only words made of pure letters, and the transformation `w.lower()` converts those words to lowercase.

Iteration 1:

`w = "Hello"`

- Check: `"Hello".isalpha()` → True (only letters).
- Transformation: `"Hello".lower()` → `"hello"`.
- Add `"hello"` to result.

Iteration 2:

`w = "World123"`

- Check: `"World123".isalpha()` → False (contains digits).
- Skip.

Iteration 3:

`w = "Python"`

- Check: `"Python".isalpha()` → True (only letters).
- Transformation: `"Python".lower()` → `"python"`.
- Add `"python"` to result.

Iteration 4:

`w = "AI!"`

- Check: `"AI!".isalpha()` → False (contains punctuation !).
- Skip.

Step 3: Collect results. `result = ["hello", "python"]`.

Step 4: Print output. `print(result)` produces the final output shown below.

8.2 Final Output

```
1 print(result)
2 # Output: ['hello', 'python']
```

Only alphabetic words are kept, and they are converted to lowercase.

Problem 9. Sort Words by Length (Descending)

9.1 Code

```
1 sentence = "Sort these words by length"
2 result = sorted(sentence.split(), key=len, reverse=True)
3 print(result)
```

Step-by-Step Trace

Step 1: Input sentence. `sentence = "Sort these words by length".`

Step 2: Split into words.

```
sentence.split() → ["Sort", "these", "words", "by", "length"]
```

Step 3: Sorting logic. We use `sorted(words, key=len, reverse=True)`: `key=len` sorts by word length, and `reverse=True` places the longest words first (descending order).

Word lengths:

- `"Sort"` → 4
- `"these"` → 5
- `"words"` → 5
- `"by"` → 2
- `"length"` → 6

Sorting by length (descending):

1. `"length"` (6)
2. `"these"` (5)
3. `"words"` (5)
4. `"Sort"` (4)
5. `"by"` (2)

Step 4: Collect results. `result = ["length", "these", "words", "Sort", "by"].`

Step 5: Print output. `print(result)` produces the final output shown below.

9.2 Final Output

```
1 print(result)
2 # Output: ['length', 'these', 'words', 'Sort', 'by']
```

The sentence is split into words, and the words are sorted by length in descending order.

Problem 10. First-and-Last-Letter Abbreviation

10.1 Code

```
1 words = ["apple", "hi", "a", "banana"]
2 result = [w[0] + w[-1] for w in words if len(w) >= 2]
3 print(result)
```

Step-by-Step Trace

Step 1: Input list. `words = ["apple", "hi", "a", "banana"]`.

Step 2: List comprehension. We loop through each word `w`. The condition `len(w) >= 2` keeps only words with at least two characters, and the transformation `w[0] + w[-1]` forms an abbreviation using the first and last characters.

Iteration 1:

`w = "apple"`

- Length: `len("apple") = 5` $\rightarrow \geq 2 \rightarrow$ condition passes.
- Abbreviation: `"a" + "e"  $\rightarrow$  "ae".`
- Add `"ae"` to `result`.

Iteration 2:

`w = "hi"`

- Length: `len("hi") = 2` $\rightarrow \geq 2 \rightarrow$ condition passes.
- Abbreviation: `"h" + "i"  $\rightarrow$  "hi".`
- Add `"hi"` to `result`.

Iteration 3:

`w = "a"`

- Length: `len("a") = 1` $\rightarrow < 2 \rightarrow$ condition fails.
- Skip.

Iteration 4:

`w = "banana"`

- Length: `len("banana") = 6` $\rightarrow \geq 2 \rightarrow$ condition passes.
- Abbreviation: `"b" + "a"  $\rightarrow$  "ba".`
- Add `"ba"` to `result`.

Step 3: Collect results. `result = ["ae", "hi", "ba"]`.

Step 4: Print output. `print(result)` produces the final output shown below.

10.2 Final Output

```
1 print(result)
2 # Output: ['ae', 'hi', 'ba']
```

Words shorter than two characters are ignored, and the remaining words are abbreviated using their first and last characters.